

views::maybe and views::nullable

Steve Downey

Abstract

Two new views of zero or one element

`views::nullable`

an adaptor over a nullable

`views::maybe`

an owning view of zero or one elements

nullable

1. Contextually convertible to `bool`
2. Dereferenceable

Things like pointers, `std::optional`, `std::expected`

Unsafe at any speed

Only safely dereferenceable if **truthy**

views::nullable

Adapt a nullable by lifting from the nullable monad to the ranges monad.

(you can safely ignore the M-word)

Example

- Before

```
auto opt = possible_value();  
if (opt) {  
    // a few dozen lines ...  
    use(*opt); // is *opt Safe ?  
}
```

- After

```
for (auto&& opt : views::nullable(possible_va  
    // a few dozen lines ...  
    use(opt); // opt is Safe  
}
```

views::maybe

A range of zero or one elements

A view the same way `views::single` is

—

$O(1)$ with large C

Example

Shows up in range comprehensions for guard clauses

```
[ (x, y, z) | z <- [1..], y <- [1..z], x <- [
```


yield_if

```
inline constexpr auto yield_if = [](bool b, a
    return b ? maybe_view{std::move(x)} : may
};
```

and_then

```
inline constexpr auto and_then = [](auto&& r,  
    return decltype(r)(r)  
        | std::ranges::views::transform(std:  
        | std::ranges::views::join;  
};
```

Desugared Comprehension

```
using std::ranges::views::iota;
auto triples = and_then(iota(1), [](int z
    return and_then(iota(1, z + 1), [=](i
        return and_then(iota(x, z + 1), [
            return yield_if(x * x + y * y
                std::make_tup
        });
    });
});
```

Similar to **filter**

Flattening a range of ranges excluding the empty range operates much like `filter`.

Different trade-offs.

Easier if the condition is not a simple property of the element.

The standard library should not be overly opinionated.

Vocabulary

Useful as a return type for range code.

Provide fit and a polish

- Monadic Ops
- T&
 - No assignment from T
 - `views::maybe never` deduces a ref

Differences from Optional

Is a range

Does not support assignment from underlying

Support for T&

Because there is no assignment from T there is no question about rebind/assign-through.

Assignment from views::maybe<T&> rebinds, preserving equality behavior.

Support for `std::reference_wrapper`

There is specialized support for eliding the `get` operation to make a `maybe<std::reference_wrapper<T>>` work directly. If `T&` specialization is in place, it should be dropped and the disjunction in the concept removed.

Examples From Test Cases

Basic maybe<int>

```
int i = 7;
maybe_view<int> v1{};
ASSERT_TRUE(v1.size() == 0);

maybe_view<int> v2{i};
ASSERT_TRUE(v2.size() == 1);
for (auto i : v1)
    ASSERT_TRUE(i != i); // tautology so

for (auto i : v2)
    ASSERT_EQ(i, 7);

int s = 4;
for (auto&& i : views::maybe(s)) {
    ASSERT_EQ(i, 4);
    i = 9;
    ASSERT_EQ(i, 9);
}
ASSERT_EQ(s, 4);
```

Basic `maybe<reference_wrapper<int>>`

```
int i = 7;

maybe_view<int> v2{std::ref(i)};

for (auto i : v2)
    ASSERT_EQ(i, 7);

int s = 4;
for (auto&& i : views::maybe(std::ref(s))
    ASSERT_EQ(i, 4);
    i.get() = 9;
    ASSERT_EQ(i, 9);
}
ASSERT_EQ(s, 9);
```

Basic Nullable

```
std::optional      s{7};

for (auto i : views::nullable(s))
    ASSERT_EQ(i, 7);

nullable_view e{std::optional<int>{}};
for (int i : e)
    ASSERT_TRUE(i != i);
```

Thank You

Speaker notes